Progetto ai_text_detector

Obiettivo:
Pipele Machine learning (PML) per classificare un testo in 6 categorie: Human, DeepSeek, Grok, Claude, Gemini, ChatGPT.
Metrica di valutazione: Macro F1 (media non pesata del F1 per classe).

Dati:

- train.csv: testi con etichetta nota, usati per allenare
- test.csv: testi senza etichetta, quelli da classificare per la submission

Considerazioni:

- Dataset sbilanciato
- Gli LLM hanno stili diversi ma potrebbero sovrapporsi su testi corti o generici

Idee:

- TF-IDF + LinearSVC come baseline TF-IDF trasforma ogni testo in un vettore di numeri: ogni parola del vocabolario diventa una colonna, e il numero indica quanto quella parola è importante in quel testo. LinearSVC è un classificatore che impara a separare le classi in quello spazio numerico. Insieme sono veloci, funzionano bene su testi, e danno subito un numero di riferimento da battere.

- Feature handcrafted TF-IDF guarda *cosa* dice il testo. Queste feature guardano *come* è scritto: quanto è lungo, quante maiuscole ha, quanta punteggiatura. Un LLM che scrive frasi corte e secche si distingue da uno che scrive paragrafi lunghi anche senza guardare le parole. Segnali complementari = modello più robusto

- Router a due stadi Se dall'EDA emerge che due classi sono quasi indistinguibili per il modello generalista, ha senso allenare un secondo modello dedicato solo a quelle due, con feature più specifiche. Il primo modello smista il traffico, il secondo risolve il caso difficile.

Possibili problemi:

- Imbalance Se il 63% dei testi è Human e solo il 3% è DeepSeek, un modello stupido che dice sempre "Human" ha già accuratezza 63% senza imparare nulla. class_weight='balanced' fa sì che gli errori sulle classi rare pesino di più durante l'allenamento, forzando il modello a imparare anche quelle.

- Testi corti TF-IDF funziona contando parole. Se il testo è una sola frase, ci sono pochi dati su cui lavorare e il segnale è debole. In quei casi le feature strutturali (lunghezza, stile) diventano l'unica informazione affidabile.

- Gap OOF/LB L'OOF è calcolato sul training set, il LB è calcolato sul test set di Kaggle. Se i due set hanno distribuzioni diverse, il numero OOF non riflette il numero reale. È normale che ci sia un gap — l'importante è che non sia enorme.

- Data leakage Nello specialista, se alleni su tutti i dati e poi valuti sugli stessi, il modello ha "già visto le risposte" e sembra più bravo di quello che è. La CV fold-per-fold garantisce che per ogni fold lo specialista venga allenato solo sui dati che *non* sta valutando.

Piano:

Fase 0: Setup ambiente Kaggle
Fase 1: EDA + Duplicate Check
Fase 2: Preprocessing (3 varianti da confrontare)
Fase 3: Baseline (3 pipeline parallele)
Fase 4: Valutazione OOF approfondita
Fase 5: Submission

Fase 6: Ottimizzazione + Error Analysis
Fase 6.0: Freeze Baseline
Fase 6.1: Riallineamento Preprocessing
Fase 6.2: Tuning Parametrico (C)
Fase 6.3: Feature Metriche .
Fase 6.4: Router Binario (DS vs Grok)
Fase 6.5: DeBERTa + Ensemble

Fase 7: Submission Finale

Note:
Fase 6.1 fallita, dunque salto fase 6.2 vado diretto alla 6.3.
Inoltre fase 6.5 non svolta perche 6.4 ha data punteggio sodddifacente.

Diario passaggi:

FASE 0: Setup ambiente Kaggle

1. Sono andato su kaggle.com e ho fatto l'accesso al mio account.
2. Ho cercato la competizione MALTO, ho cliccato su "Join Competition" e ho accettato le regole.
3. Ho cliccato su "New Notebook" per creare il mio ambiente Jupyter nel browser, assicurandomi di avere Python 3 e la GPU disponibile.
4. Dalla sidebar laterale ho cliccato su "Add Input" → “Competition Data” per aggiungere i dati della competizione al mio spazio di lavoro.
5. Ho verificato i percorsi eseguendo import os e os.listdir('/kaggle/input/competitions/malto-recruitment-hackathon/'), confermando la presenza dei file di addestramento e test.

   Output: ['train.csv', 'test.csv']

FASE 1: EDA + Duplicate Check Passaggi eseguiti:

1. Ho importato le librerie: pandas, numpy, matplotlib, seaborn, re, sklearn.
2. Ho caricato train.csv e test.csv con pd.read_csv() e controllato con .head() e .info(). Output: Train = 2400 righe, colonne TEXT e LABEL. Test = 600 righe, colonne Unnamed: 0 e TEXT.
3. Ho controllato i valori mancanti con .isnull().sum(). Output: nessun valore mancante né in train né in test.
4. Ho analizzato la distribuzione delle label con .value_counts() e un grafico a barre. Output: dataset fortemente sbilanciato. Human=1520 (63%), ChatGPT=320, Gemini=240, Grok=160, DeepSeek=80, Claude=80. Nota: class_weight='balanced' sarà obbligatorio nei modelli.
5. Ho eseguito il duplicate check: duplicati esatti, duplicati dopo normalizzazione spazi, testi uguali con label diverse. Output: 0 duplicati esatti, 0 dopo normalizzazione, 0 conflitti. Dataset pulito.
6. Ho confrontato la distribuzione delle lunghezze (in caratteri) tra train e test con un istogramma sovrapposto. Output: le distribuzioni si sovrappongono bene, nessun bias evidente.
7. Ho calcolato lunghezza media in caratteri e parole per classe con groupby + boxplot. Output: DeepSeek (560 car.) e Grok (414 car.) sono i più corti — segnale discriminativo forte. Claude è il più lungo (3358 car.).
8. Ho calcolato feature stilistiche per classe: densità punteggiatura, % maiuscole, densità markdown. Output: Human ha molte più maiuscole (0.055 vs ~0.01 degli AI). ChatGPT ha pochissime maiuscole extra (0.004). Claude e ChatGPT hanno più punteggiatura.
9. Ho letto 2 esempi per classe a occhio. Nota: Human ha errori di battitura e stile informale. DeepSeek/Grok hanno frasi brevi e stile enciclopedico. Claude usa frasi molto corte separate da punti. ChatGPT ha tono accademico e vocabolario ricercato.
10. Ho annotato tutte le osservazioni in una cella Markdown nel notebook.

Fase 2: Preprocessing (3 Varianti)

1. Ho definito 3 funzioni di pulizia del testo: raw_text (solo normalizzazione spazi multipli), lower_text (solo lowercase), clean_text (lowercase + rimozione URL + rimozione caratteri non stampabili + normalizzazione spazi). Nessuna delle 3 rimuove la punteggiatura, in quanto è un segnale discriminativo fondamentale per questo task.
2. Ho applicato le 3 funzioni a train e test con .apply(), creando 3 nuove colonne per ciascun DataFrame (raw_text, lower_text, clean_text). La colonna TEXT originale non è stata sovrascritta.
3. Ho eseguito il controllo visivo su un esempio (riga 0, classe Human): confermato che la punteggiatura è intatta in tutte e 3 le varianti e che le trasformazioni sono corrette.
4. Ho controllato i testi vuoti dopo clean_text. Output: 0 testi vuoti in train, 0 in test. Dataset integro. Nota: l'esempio ispezionato visivamente era un testo Human, riconoscibile dagli errori di battitura (ilves, citizns, votingg, sohuld) — coerente con le osservazioni dell'EDA.

Fase 3: Baseline (3 pipeline parallele)

1. Definite 3 pipeline con LinearSVC(class_weight='balanced', max_iter=2000): P1 (Word TF-IDF, max_features=50000, ngram_range=(1,2), sublinear_tf=True), P2 (Char TF-IDF, analyzer='char_wb', ngram_range=(3,5), max_features=100000, sublinear_tf=True), P3 (FeatureUnion Word+Char).
2. Valutazione OOF con cross_val_predict, StratifiedKFold 5 fold, random_state=42, n_jobs=-1.
3. Risultati Macro F1: P1=0.8887, P2=0.8787, P3=0.8886. Pipeline scelta: P1 (Word TF-IDF) — migliore e più veloce. P3 non supera P1: i char n-grams non aggiungono informazione sufficiente su questo dataset.
4. Mappatura label scoperta ispezionando esempi: 0=Human, 1=DeepSeek, 2=Grok, 3=Claude, 4=Gemini, 5=ChatGPT.
5. Classification report P1: Human F1=1.00, Claude F1=1.00, Gemini F1=0.99, ChatGPT F1=0.99, Grok F1=0.85, DeepSeek F1=0.50 (recall=0.36). DeepSeek è il problema principale.
6. Confusion matrix confermata: 47/80 testi DeepSeek classificati come Grok. Causa: entrambi hanno stile enciclopedico e testi corti. Claude perfetto nonostante 80 esempi: stile unico (frasi cortissime separate da punti) già identificato nell'EDA.

Fase 4: Valutazione OOF approfondita

1. Definita oof_preds = oof_p1 per allineare il nome della variabile con il codice della Fase 4. Verificate le variabili con print: train shape (2400, 10), oof_preds lunghezza 2400, y_train lunghezza 2400. Tutto coerente.
2. Generata confusion matrix visuale con ConfusionMatrixDisplay e cmap='Blues'. Output: Human 1520/1520, Claude 80/80, Gemini 238/240, ChatGPT 318/320, Grok 152/160, DeepSeek 29/80. Problema principale confermato visivamente: 47 testi DeepSeek classificati come Grok.
3. Stampato classification report completo per classe. Output: Human F1=1.00, Claude F1=1.00, Gemini F1=0.99, ChatGPT F1=0.99, Grok F1=0.85, DeepSeek F1=0.50 (recall=0.36). Macro F1=0.8887.
4. Creato DataFrame degli errori filtrando i campioni dove LABEL ≠ oof_pred. Output: 63 errori totali su 2400 (2.6%). Distribuzione: DeepSeek=51, Grok=8, Gemini=2, ChatGPT=2.
5. Letti 3 esempi di testi DeepSeek classificati come Grok. Output: i testi sbagliati sono frasi singole brevissime (es. "A star's life cycle begins in a nebula...", "Nuclear fission releases immense energy...", "Aged Gouda from the Netherlands..."). Nota: il problema non è che DeepSeek e Grok scrivono uguale — è che i DeepSeek più corti non hanno abbastanza testo per far emergere segnali discriminativi nel TF-IDF.
6. Annotate tutte le osservazioni in cella Markdown nel notebook.

Fase 5: Submission

1. Ho verificato con os.walk() che sample_submission.csv non fosse presente nella cartella della competizione. Output: solo train.csv e test.csv disponibili. Nota: nessuna violazione — il sample_submission è solo un template di formato, ricavabile direttamente dal test set.
2. Ho riaddestrato pipe_p1 su tutti i 2400 campioni con pipe_p1.fit(X_train, y_train), usando X_train = train['clean_text'] (stessa variante usata dalla Fase 3). Output: "Modello addestrato su tutti i dati ✓".
3. Ho preparato X_test = test['clean_text'] e generato le predizioni con pipe_p1.predict(X_test). Output: 600 predizioni. Distribuzione predetta: Human(0)=380, DeepSeek(1)=11, Grok(2)=47, Claude(3)=20, Gemini(4)=61, ChatGPT(5)=81. Nota: Human=380/600=63.3%, coerente con il training (1520/2400=63.3%) — segnale positivo di consistenza del modello.
4. Ho costruito il DataFrame di submission con ID = test['Unnamed: 0'] e LABEL = y_pred.astype(int). Esportato in submission.csv con index=False. Verificato rileggendo con pd.read_csv(): shape (600, 2), tipi corretti.
5. Ho caricato submission.csv su Kaggle tramite Submit Predictions. Output: Macro F1 pubblico = 0.86286. Il gap rispetto all'OOF (0.8887) è fisiologico — il test set contiene dati mai visti. Il problema principale rimane DeepSeek (recall basso)

Ottimizzazione + Error Analysis

Fase 6.0: Freeze Baseline

1. Ho aperto il notebook Kaggle su cui stavo lavorando (quello con le pipeline P1/P2/P3 già eseguite) e ho aggiunto una nuova cella in fondo.
2. Ho eseguito import shutil e poi shutil.copy("submission.csv", "submission_baseline.csv") per creare una copia del file di submission attuale, così da avere sempre un fallback al LB 0.86286.
3. Ho verificato che il file submission_baseline.csv fosse presente nella working directory controllando con os.listdir('.').
4. Ho creato la tabella degli esperimenti inizializzando una lista results e aggiungendo il dizionario della baseline con i valori noti: oof=0.8887, f1_ds=0.50, f1_grok=0.85, submitted=True.
5. Ho stampato la tabella con pd.DataFrame(results) per confermare che la riga baseline fosse registrata correttamente.
6. Ho rinominato il notebook in baseline_locked cliccando sul titolo in alto a sinistra, poi ho salvato una versione permanente tramite "Save Version" → "Save & Run All". Questo snapshot non verrà più toccato.
7. Ho duplicato il notebook (⋯ → "Copy") e rinominato la copia malto_fase6. Da questo punto in poi tutte le modifiche avvengono solo sulla copia.

Fase 6.1: Riallineamento Preprocessing

1. Nella cella 19 sostituito X_train = train['clean_text'] con i 3 input corretti: X_train = train['clean_text'] (per compatibilità celle vecchie), X_low = train['lower_text'] (per P1 word), X_raw = train['raw_text'] (per P2 char), più X_low_test e X_raw_test per il test set.
1. Aggiunta nuova cella in fondo con le 3 pipeline v2: P1 su lower_text, P2 su raw_text, P3 con loop manuale fold-per-fold combinando word(lower) + char(raw).
2. Eseguito OOF 5-fold su tutte e 3. Risultati: P1 v2 = 0.8862, P2 v2 = 0.8787, P3 v2 = 0.8886. F1 DeepSeek invariato a 0.50. Nessun miglioramento rispetto al baseline (0.8887). Il preprocessing non era il bottleneck — il problema è strutturale: DeepSeek ha testi troppo corti per il TF-IDF.

Fase 6.2: Tuning Parametrico → Eliminata per fallimento fase 6.1

Fase 6.3: Feature Metriche

1. Definita la funzione calc_features(df) e applicata a train e test. Le 6 feature calcolate: text_len, word_count, sent_count, avg_w_sent, punct_density, upper_ratio.
2. Stampata la media per classe. Valori chiave per il router: DeepSeek text_len=560, word_count=83, sent_count=4.5 — Grok text_len=414, word_count=55, sent_count=3.5. Simili ma distinguibili.
3. Feature pronte in memoria su train e test.

Fase 6.4: Router Binario (DS vs Grok)

1. Allenato il modello principale (Word TF-IDF + LinearSVC) su clean_text su tutti i 2400 campioni.
2. Allenato uno specialista binario solo sui 240 campioni con LABEL in [1,2] (DeepSeek e Grok). Input: word TF-IDF su lower_text + char TF-IDF su raw_text + 6 feature metriche scalate con StandardScaler, combinati con sparse.hstack.
3. Valutato lo specialista OOF su [1,2]: DeepSeek F1=0.64, Grok F1=0.86. Corretti 42/80 DeepSeek.
4. Scoperto data leakage nella prima versione del codice (OOF gonfiato a 0.9928 — il specialista vedeva campioni già usati nel training). Corretto con loop OOF fold-per-fold: per ogni fold, specialista allenato solo sui [1,2] del fold train e applicato solo sul fold validation.
5. OOF corretto con router: Macro F1 = 0.9165 (baseline era 0.8887). F1 DeepSeek = 0.64, F1 Grok = 0.87. Nessun crollo su altre classi.
6. Generata submission_router.csv. Distribuzione predizioni test: Human=380 (63.3%), DeepSeek=14, Grok=44, Claude=20, Gemini=61, ChatGPT=81. Human% coerente col training (63.3%). Pronta per submission.

Fase 6.5: DeBERTa + Ensemble → Eliminata risultato 6.4 soddisfacente

Fase 6.6: Submission Finale

1. Aggiornata variabile OOF nella checklist finale: sostituito oof_preds con oof_global (predizioni del router dalla Fase 6.4). Il nome sbagliato puntava ancora alle predizioni della Fase 3.
2. Eseguita checklist finale. Output: OOF Macro F1 = 0.9165 ✅, F1 DeepSeek = 0.64 ✅, F1 Grok = 0.87 ✅, F1 Human = 1.00 ✅, distribuzione Human nel test = 63.3% ✅. Tutti i criteri soddisfatti.
3. Aggiornata tabella esperimenti con riga router_6.4: oof=0.9165, f1_ds=0.64, f1_grok=0.87, submitted=True. Eseguito backup: shutil.copy("submission_router.csv", "submission_fase64.csv").
4. Caricato submission_router.csv su Kaggle tramite Submit Predictions. Output: Macro F1 pubblico = 0.90515. Miglioramento di +0.042 rispetto alla baseline (0.86286). Fase 6.5 non eseguita — OOF 0.9165 supera la soglia trigger di 0.90 prevista dal piano. Lavoro completato.

STRUTTURA:

Fase	Attività	Celle Notebook
0	Setup dell'ambiente di lavoro	1
1	Analisi Esplorativa dei Dati (EDA) e Verifica Duplicati	2, 3, 4, 5, 6, 7
2	Preprocessing dei Dati	8, 9
3	Implementazione del Modello Baseline	10
4	Valutazione Approfondita Out-Of-Fold (OOF)	11, 13
5	Sottomissione Iniziale (Baseline)	12
6	Ottimizzazione del Modello (Router)	14
7	Sottomissione Finale e Verifica di Coerenza (Sanity Check)	15, 16

Filo Logico:

1. Setup librerie [1]
2. Carica gli strumenti [2]
3. Carica i dati: 2400 train, 600 test [3]
4. Sbilanciamento [4]
5. Verifica: no duplicati, no conflitti di label [5]
6. Guarda i testi a occhio: stili diversi [6]
7. Scoperta: lunghezza testo discriminativa per classe [7]
8. Scoperta: maiuscole e punteggiatura variano per classe [8]
9. Crea 3 varianti di testo (raw/lower/clean) per facilitare feature numeriche [9]
10. Crea 6 feature numeriche strutturali (contare le caratteristiche fisiche della frase)[10]
11. Baseline: TF-IDF word + LinearSVC → OOF [11]
12. Analisi: il problema è quasi solo DeepSeek ↔ Grok (testi corti) [12]
13. Salva submission baseline → caricata su Kaggle [13]
14. Tabella comparativa dei risultati [14]
15. Soluzione: router a due stadi con specialista DS/Grok [15]
16. Applica router e salva submission finale [16]

Approfondimenti:

10 → 6 feature numeriche strutturali

- text_len (Lunghezza del testo)
- word_count (Conteggio parole)
- sent_count (Conteggio frasi)
- avg_w_sent (Media parole per frase)
- punct_density (Densità punteggiatura)
- upper_ratio (Percentuale di maiuscole)

11 → Baseline: TF-IDF word + LinearSVC → OOF

Il "TF-IDF": prende ogni parola usata e la trasforma in un numero, creando una sorta di codice a barre per ogni frase.

Il "LinearSVC": guarda questi codici a barre e tira una linea matematica per separare i codici degli Umani da quelli dei Robot.

“OFF” test con testi mai visti prima

15 → router a 2 stai

Il primo programma legge e smista subito al volo tutti i testi facili. Se però si trova davanti un testo cortissimo

Secondo "specialista", allenato esclusivamente per distinguere i testi difficili (DeepSeek e Grok)

Spiegazione dettagliata:

FASE 0 — Setup ambiente

Cella 1 — Import librerie e costanti

import re, numpy, pandas, matplotlib, seaborn, scipy, sklearn...

LABEL_NAMES = ['Human', 'DeepSeek', 'Grok', 'Claude', 'Gemini', 'ChatGPT']

DATA_DIR = '/kaggle/input/...'

Cosa fa:
Questa cella carica tutte le librerie che serviranno nel notebook e definisce alcune costanti di base. È il punto di partenza dell'intero progetto: prima di lavorare sui dati, bisogna preparare tutti gli strumenti necessari.

Perché è importante:
Senza questa cella, le successive non funzionerebbero, perché ogni parte del pipeline dipende da librerie specifiche per leggere dati, trasformare testo, allenare modelli, valutare risultati e creare grafici.

Perché ogni libreria viene usata:

- **re**: serve per le espressioni regolari, quindi per cercare, rimuovere o sostituire pattern nel testo, ad esempio URL, spazi multipli o caratteri indesiderati.

- **numpy**: è la base del calcolo numerico in Python. Molti algoritmi di machine learning lavorano internamente su array numpy.

- **pandas**: permette di gestire i dati in forma tabellare tramite i DataFrame. I file CSV vengono caricati proprio in questa struttura.

- **matplotlib** e **seaborn**: servono per visualizzare i dati. Matplotlib è il motore principale, mentre seaborn rende i grafici più leggibili e curati.

- **scipy.sparse**: è fondamentale quando si lavora con TF-IDF, perché le matrici testuali hanno moltissime colonne ma quasi tutte con valore zero. Le matrici sparse salvano memoria conservando solo i valori utili.

- **TfidfVectorizer**: converte il testo in numeri, cioè in feature utilizzabili dal modello.

- **LinearSVC**: è il classificatore scelto per distinguere le 6 classi.

- **Pipeline**: permette di unire più passaggi in un unico oggetto, ad esempio trasformazione TF-IDF + classificatore.

- **StratifiedKFold**: divide i dati in fold mantenendo la stessa proporzione tra le classi.

- **cross_val_predict**: esegue la cross-validation e restituisce le predizioni out-of-fold.
- **StandardScaler**: normalizza le feature numeriche per evitare che alcune pesino troppo solo perché hanno valori più grandi.
- **f1_score**, **classification_report**, **confusion_matrix**: servono per valutare le performance del modello.

Costanti definite:

- **LABEL_NAMES**: è una mappa tra i numeri delle etichette e i nomi reali delle classi. Il dataset usa etichette numeriche da 0 a 5, ma questa lista consente di stampare nomi leggibili come “DeepSeek” o “Claude”.
- **DATA_DIR**: indica il percorso in cui si trovano i file CSV su Kaggle.

FASE 1 — EDA + Duplicate Check

EDA significa *Exploratory Data Analysis*, cioè analisi esplorativa dei dati.
Prima di costruire un modello, bisogna capire bene che tipo di dati si stanno usando.

Cella 2 — Caricamento dati

```
train = pd.read_csv(DATA_DIR + 'train.csv')

test  = pd.read_csv(DATA_DIR + 'test.csv')

print(f"Train: {train.shape}  |  null: ...")
```

Cosa fa:
Legge i file `train.csv` e `test.csv` e li carica in due DataFrame pandas. Poi stampa la dimensione dei dataset e controlla se ci sono valori mancanti.

Perché è importante:
Questa cella serve a verificare subito che i dati siano stati caricati correttamente e che non ci siano problemi evidenti. In particolare, è fondamentale controllare eventuali valori nulli, perché una colonna di testo con valori mancanti potrebbe far fallire il preprocessing o il training.

Interpretazione tipica dell’output:

- **Train: (2400, X)** → ci sono 2400 testi con etichetta già nota, usati per allenare e valutare il modello.

- **Test: (600, X)** → ci sono 600 testi senza etichetta, che il modello dovrà classificare per generare la submission finale.

Cella 3 — Distribuzione delle classi

train['LABEL'].value_counts().sort_index()

plt.bar(...)

Cosa fa:
Conta quanti esempi ci sono in ciascuna classe e visualizza la distribuzione con un grafico a barre.

Perché è importante:
Questa cella permette di capire se il dataset è bilanciato oppure no. Nel nostro caso emerge subito che il dataset è fortemente sbilanciato: la classe **Human** è dominante, mentre classi come **DeepSeek** e **Claude** sono molto più rare.

Insight principale:

- **Human**: circa 1520 esempi, cioè circa il 63% del dataset
- **DeepSeek**: circa 80 esempi
- **Claude**: circa 80 esempi

Questo è un punto critico, perché un modello potrebbe limitarsi a predire sempre "Human" e ottenere comunque una buona accuratezza, senza aver davvero imparato a distinguere le classi. Per questo motivo, più avanti si usa `class_weight='balanced'`, che penalizza maggiormente gli errori sulle classi rare.

Cella 4 — Duplicate Check

dup_exact = train.duplicated(subset=['TEXT']).sum()

norm = train['TEXT'].str.lower().str.replace(r'\s+', ' ', ...)

dup_norm = norm.duplicated().sum()

conflicts = train.groupby(norm)['LABEL'].nunique()

Cosa fa:
Controlla la presenza di duplicati nel training set in tre modi diversi:

1. cerca testi identici al 100%;

2. cerca testi uguali dopo una normalizzazione semplice, cioè trasformazione in minuscolo e uniformazione degli spazi;

3. verifica se esistono testi uguali ma associati a etichette diverse.

Perché è importante:
Questa analisi serve a evitare problemi di qualità nei dati.

- I **duplicati esatti** possono gonfiare artificialmente le metriche.

- I **duplicati nascosti** possono non essere rilevati a occhio, ma introdurre lo stesso problema.

- I **conflitti di label** sono ancora più gravi: se lo stesso testo compare con etichette diverse, il modello riceve segnali contraddittori e impara peggio.

In pratica, questa cella controlla se il dataset è coerente e affidabile prima di procedere con l'addestramento.

Cella 5 — Distribuzione delle lunghezze dei testi

plt.hist(train['TEXT'].str.len(), ...)

stats = train.assign(text_len=..., word_count=...).groupby('LABEL').mean()

Cosa fa:
Visualizza un istogramma della lunghezza dei testi e calcola alcune statistiche medie per classe, come numero di caratteri e numero di parole.

Perché è importante:
La lunghezza del testo si rivela subito una feature molto informativa. Alcuni modelli generano testi molto brevi, altri molto lunghi. Questo significa che il modello può già distinguere parte delle classi osservando la struttura del testo, ancora prima di analizzarne il contenuto.

Insight emersi:

- **DeepSeek**: testi molto corti

- **Grok**: anch'esso tende a testi corti

- **Claude**: testi molto lunghi

- **Human**: lunghezza intermedia ma con alta variabilità

Conseguenza pratica:
Questa osservazione giustifica l'introduzione di feature numeriche come `text_len` nella fase successiva. Inoltre spiega perché DeepSeek è difficile da classificare: testi più brevi significano meno informazioni disponibili.

Cella 6 — Feature stilistiche

```
def stylistic_features(text):

  return pd.Series({

      'punct':    len([c for c in text if c in '.,;:!?']) / n,

      'upper':    sum(c.isupper() for c in text) / n,

      'markdown': len(re.findall(r'[#\*\-]', text)) / n,

  })
```

Cosa fa:
Calcola, per ogni testo, alcune feature stilistiche semplici ma utili:

- densità di punteggiatura;
- rapporto di lettere maiuscole;
- frequenza di simboli markdown.

Perché è importante:
Il contenuto semantico non è l'unico elemento utile. Ogni modello tende anche ad avere uno stile di scrittura riconoscibile. Alcuni usano più punteggiatura, altri meno; alcuni scrivono in modo molto uniforme, altri mostrano più variazioni.

Risultati interpretativi:

- **Human** tende ad avere più maiuscole, magari per errori di digitazione, acronimi o enfasi.
- **ChatGPT** tende ad avere pochissime maiuscole fuori standard, quindi appare molto pulito e regolare.
- **Claude** e **ChatGPT** mostrano spesso più punteggiatura, coerente con uno stile formale e ordinato.

Queste osservazioni motivano l'inclusione di feature come `upper_ratio` e `punct_density` nella fase di feature engineering.

Cella 7 — Esempi di testi per classe

```
for label, name in enumerate(LABEL_NAMES):

  for text in train[train['LABEL'] == label]['TEXT'].head(2):

      print(text[:300])
```

Cosa fa:
Mostra alcuni esempi reali di testo per ciascuna classe.

Perché è importante:
Guardare i dati con i propri occhi è indispensabile. Le statistiche aiutano, ma leggere esempi concreti permette di cogliere sfumature che i numeri da soli non mostrano.

Cosa si osserva tipicamente:

- **Human**: più errori, tono colloquiale, punteggiatura meno regolare
- **DeepSeek / Grok**: testi brevi, spesso enciclopedici o molto sintetici
- **Claude**: risposte più lunghe e articolate, con frasi separate e uso di liste
- **ChatGPT**: stile uniforme, vocabolario ricercato, tono accademico

Questa cella è importante perché orienta tutte le scelte successive di preprocessing e modellazione.

FASE 2 — Preprocessing

Cella 8 — Tre varianti di testo

```
def raw_text(text):   # solo normalizza spazi

def lower_text(text): # solo lowercase

def clean_text(text): # lowercase + rimuove URL + rimuove caratteri strani
```

Cosa fa:
Crea tre versioni diverse dello stesso testo:

- **raw_text**: conserva quasi tutto, limitandosi a normalizzare gli spazi;
- **lower_text**: trasforma il testo in minuscolo ma mantiene il resto;

- **clean_text**: rende il testo più pulito, togliendo URL e rumore.

Perché è importante:
Non tutte le componenti del sistema hanno bisogno dello stesso livello di pulizia. Alcune informazioni, come le maiuscole, possono essere utili per riconoscere lo stile. Altre volte conviene rimuovere il rumore per concentrarsi solo sul contenuto.

Uso delle tre varianti:

- **raw_text** → utile quando si vogliono preservare i dettagli stilistici, come nel char TF-IDF
- **lower_text** → utile quando interessa il contenuto ma non la distinzione tra maiuscole e minuscole
- **clean_text** → utile per il modello principale, che deve lavorare su testo più pulito e stabile

Esempio concreto:

Testo originale:
`"Hello! Check http://example.com — it's GREAT"`

- **raw_text** → `"Hello! Check http://example.com — it's GREAT"`
- **lower_text** → `"hello! check http://example.com — it's great"`
- **clean_text** → `"hello! check it's great"`

Alla fine si controlla anche che nessun testo diventi vuoto dopo la pulizia, perché una stringa vuota causerebbe problemi nelle fasi successive.

Cella 9 — Feature handcrafted

```
def calc_features(df):
    df['text_len']      = df['TEXT'].str.len()
    df['word_count']    = df['TEXT'].str.split().str.len()
    df['sent_count']    = df['TEXT'].str.count(r'[.!?]') + 1
    df['avg_w_sent']    = df['word_count'] / df['sent_count']
    df['punct_density'] = ...
```

```
df['upper_ratio']   = ...
```

Cosa fa:
Aggiunge al dataset alcune feature numeriche create manualmente a partire dal testo.

Perché è importante:
Il TF-IDF cattura il contenuto lessicale, cioè quali parole compaiono. Queste feature numeriche invece descrivono la struttura del testo: lunghezza, numero medio di parole per frase, densità di punteggiatura, uso delle maiuscole. Sono segnali diversi ma complementari.

Significato delle principali feature:

- **text_len**: lunghezza totale del testo in caratteri
- **word_count**: numero di parole
- **sent_count**: numero di frasi stimato
- **avg_w_sent**: numero medio di parole per frase
- **punct_density**: quantità di punteggiatura rispetto alla lunghezza
- **upper_ratio**: rapporto di lettere maiuscole

Perché `avg_w_sent` è utile:
Alcuni modelli producono frasi molto brevi, altri frasi molto lunghe. Questa feature sintetizza bene questa differenza stilistica.

`feat_cols`:
Più avanti viene creata una lista con i nomi di queste colonne, così da poter selezionare rapidamente solo le feature numeriche da concatenare alle altre.

FASE 3 — Baseline

Cella 10 — Pipeline baseline + Cross-Validation OOF

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)



pipe_p1 = Pipeline([

  ('tfidf', TfidfVectorizer(max_features=50000, ngram_range=(1,2), sublinear_tf=True)),

  ('clf',   LinearSVC(class_weight='balanced', max_iter=2000))
```

])

oof_p1 = cross_val_predict(pipe_p1, train['clean_text'], y_train, cv=cv, n_jobs=-1)

Cosa fa:
Costruisce la pipeline baseline, cioè il primo modello completo, e la valuta con cross-validation stratificata usando predizioni out-of-fold.

Spiegazione dei componenti

TF-IDF

Trasforma i testi in vettori numerici.

Come funziona:

- **TF (Term Frequency)** misura quanto spesso una parola compare in un testo;
- **IDF (Inverse Document Frequency)** riduce il peso delle parole troppo frequenti in tutti i documenti, perché non aiutano a distinguere le classi.

Quindi TF-IDF valorizza le parole informative e penalizza quelle troppo comuni.

Parametri importanti:

- **max_features=50000**: limita il vocabolario alle 50.000 feature più utili
- **ngram_range=(1,2)**: usa sia unigrammi che bigrammi
- **sublinear_tf=True**: applica una trasformazione logaritmica alle frequenze, per evitare che i testi più lunghi dominino troppo

---

LinearSVC

È il classificatore lineare usato per separare le classi nello spazio delle feature TF-IDF.

Parametri importanti:

- **class_weight='balanced'**: compensa lo sbilanciamento tra le classi
- **max_iter=2000**: aumenta il numero massimo di iterazioni per garantire convergenza

---

StratifiedKFold

Divide il training set in 5 fold mantenendo le proporzioni delle classi in ogni fold.

Perché è importante:
Evita che una fold contenga troppe o troppo poche istanze di una certa classe, rendendo la valutazione più affidabile.

Out-of-Fold (OOF)

Le predizioni OOF sono una stima onesta delle performance.

Come funziona:

1. il training set viene diviso in 5 fold;
2. a turno, 4 fold vengono usati per allenare e 1 per validare;
3. ogni esempio riceve una predizione solo quando si trova nel fold di validazione, quindi da parte di un modello che non l'ha mai visto in training.

Perché è importante:
Valutare sullo stesso dataset usato per allenare porterebbe a risultati troppo ottimistici. Le predizioni OOF simulano molto meglio il comportamento reale sul test set.

FASE 4 — Valutazione OOF approfondita

Cella 11 — Confusion matrix + Error analysis

```
cm = confusion_matrix(y_train, oof_p1)

ConfusionMatrixDisplay(cm, ...).plot(...)


errors  = oof_df[oof_df['LABEL'] != oof_df['pred']]

ds_grok = errors[(errors['LABEL'] == 1) & (errors['pred'] == 2)]
```

Cosa fa:
Calcola la confusion matrix e analizza gli errori del modello, con attenzione particolare alle confusioni tra classi simili.

Cos'è la confusion matrix:
È una matrice in cui:

- le **righe** rappresentano le classi reali;
- le **colonne** rappresentano le classi predette;
- la **diagonale** contiene le predizioni corrette;
- le celle fuori diagonale mostrano gli errori.

Perché è importante:
Non basta sapere che il modello sbaglia: bisogna capire **dove** sbaglia. Questa cella mostra che il problema principale non riguarda tutte le classi allo stesso modo, ma è concentrato soprattutto nella distinzione tra **DeepSeek** e **Grok**.

Insight chiave:
Le altre classi risultano già ben separate, mentre DeepSeek e Grok vengono confusi frequentemente tra loro. Questa osservazione è fondamentale, perché suggerisce che non serve riprogettare tutto il sistema: basta costruire un modulo specializzato proprio per quella coppia di classi.

Cella 12 — Genera `submission_baseline.csv`

```
pipe_p1.fit(train['clean_text'], y_train)

y_pred = pipe_p1.predict(test['clean_text'])

pd.DataFrame({'ID': ..., 'LABEL': y_pred}).to_csv('submission_baseline.csv')
```

Cosa fa:
Allena la pipeline baseline su tutto il training set e genera le predizioni finali sul test set, salvandole in un file CSV pronto per Kaggle.

Perché è diversa dalla Cella 10:
Nella Cella 10 il training set veniva diviso in fold per stimare le performance. Qui invece si usa **tutto** il training set per ottenere il modello più forte possibile prima di predire sul test.

Perché è corretto farlo:
Una volta conclusa la fase di valutazione, per la submission finale conviene allenare il modello su tutti i dati disponibili, perché più esempi in training significano in genere un modello migliore.

Cella 13 — Tabella risultati comparativa

```
results = pd.DataFrame([
```

```
  {'nome': 'baseline',   'oof': 0.8887, 'f1_ds': 0.50, ...},

  {'nome': 'router_6.4', 'oof': 0.9165, 'f1_ds': 0.64, ...},

])
```

Cosa fa:
Crea una tabella riassuntiva con i risultati dei principali esperimenti.

Perché è utile:
Permette di confrontare rapidamente baseline e versioni migliorate senza dover rileggere tutto l'output del notebook.

Perché è hardcoded:
Non viene calcolata automaticamente, ma mantenuta a mano come registro degli esperimenti. È una scelta pratica: serve come riepilogo rapido durante lo sviluppo.

FASE 6 — Ottimizzazione con Router

Cella 14 — Router con cross-validation

Questa è la cella più complessa, perché introduce l'architettura finale a due stadi.

Idea di base:
Un solo classificatore per 6 classi funziona bene in generale, ma non è ottimale per distinguere DeepSeek da Grok. Per questo si introduce un sistema con due modelli:

1. **modello principale** → classifica tutte le 6 classi;
2. **modello specialista** → interviene solo quando il modello principale predice DeepSeek o Grok.

Schema logico del router:

Testo → Modello principale → predizione tra 6 classi

↓

se la predizione è DeepSeek o Grok

↓

Modello specialista DeepSeek/Grok

↓

predizione finale raffinata

Feature usate dallo specialista

```
Xw_f = tw_f.fit_transform(train['lower_text'])  # TF-IDF word

Xc_f = tc_f.fit_transform(train['raw_text'])    # TF-IDF char

Xm_f = sc_f.fit_transform(train[feat_cols])     # feature numeriche

X_f  = sparse.hstack([Xw_f, Xc_f, Xm_f])
```

Cosa fa:
Costruisce una rappresentazione più ricca del testo, unendo tre tipi di feature:

- **word TF-IDF** su `lower_text`: cattura il vocabolario;
- **char TF-IDF** su `raw_text`: cattura lo stile a livello di sequenze di caratteri;
- **feature numeriche**: catturano la struttura del testo.

Perché questa combinazione funziona meglio:
DeepSeek e Grok sono molto simili nel contenuto generale, quindi solo il vocabolario non basta sempre. Le differenze più utili stanno spesso nello stile, nella lunghezza e nei pattern di scrittura. Unendo tutte queste informazioni, lo specialista ha più possibilità di distinguerli.

Perché usare il char TF-IDF

Il char TF-IDF lavora su sequenze di caratteri, non su parole intere. Questo è utile perché riesce a catturare micro-pattern di stile, come suffissi, abbreviazioni, modi ricorrenti di scrivere o strutture formali.

`analyzer='char_wb'` considera sequenze di caratteri all'interno delle parole, evitando di creare n-grammi che attraversano gli spazi. È una scelta robusta per cogliere lo stile lessicale senza troppo rumore.

Perché usare StandardScaler

Le feature numeriche hanno scale molto diverse:

- `text_len` può valere migliaia;
- `upper_ratio` è compreso tra 0 e 1.

Senza normalizzazione, il modello darebbe troppo peso alle feature con numeri più grandi. Lo scaler le riporta su una scala comparabile.

Perché la cross-validation è scritta a mano

Qui non basta `cross_val_predict`, perché il router ha una logica a due stadi.

Lo specialista deve essere allenato solo sui dati di training di ciascun fold, e poi applicato ai dati di validazione del fold stesso. Questo richiede un controllo manuale dell'intero processo, per evitare leakage e per rispettare correttamente la separazione tra training e validation.

FASE 7 — Submission finale + Sanity check

Cella 15 — Allena il router finale e genera `submission_router.csv`

```
pipe_main.fit(train['clean_text'], y_train)

# ... specialista su tutti i [1,2] del train

test_preds[mask_r] = spec_final.predict(X_r)

pd.DataFrame(...).to_csv('submission_router.csv')
```

Cosa fa:
Allena il modello principale e lo specialista su tutto il training set, poi applica il router al test set reale e salva la submission finale.

Perché è importante:
Questa è la versione definitiva del sistema. Riproduce la stessa logica validata nella cross-validation, ma sfrutta tutti i dati disponibili per generare la predizione finale più forte possibile.

Cella 16 — Sanity check finale

```
oof_score = f1_score(y_train, oof_global, average='macro')

report    = classification_report(y_train, oof_global, target_names=LABEL_NAMES,
output_dict=True)
```

```
print(f"OOF Macro F1:  {oof_score:.4f}  {'✅' if oof_score >= 0.8887 else '❌'}")

print(f"F1 DeepSeek:   {report['DeepSeek']['f1-score']:.2f} ...")

print(f"F1 Grok:       {report['Grok']['f1-score']:.2f} ...")

print(f"F1 Human:      {report['Human']['f1-score']:.2f} ...")


sub       = pd.read_csv('submission_router.csv')

human_pct = sub['LABEL'].value_counts(normalize=True).get(0, 0)

print(f"Human nel test: {human_pct:.1%}  (atteso ~63%)")
```

Cosa fa:
Esegue una serie di controlli finali per verificare che il modello sia coerente prima di caricare la submission su Kaggle.

Controlli principali:

- l'OOF Macro F1 deve essere almeno pari o superiore alla baseline;
- la F1 di DeepSeek deve migliorare;
- la F1 di Grok non deve peggiorare troppo;
- la F1 di Human deve restare molto alta;
- la percentuale di predizioni Human nel test deve rimanere vicina a quella osservata nel train.